Database dependencies

Not all field options work as advertised, due to underlying database implementations. For strings containing ASCII, SQLite only does case-insensitive matching for substrings. For strings containing values outside the ASCII range, SQLite only does exact matching. Aren't computers fun? For a full list of the peculiarities of Django interacting with your database of choice, see http://mng.bz/ma28.

There are many more field look-ups built into Django, including one for querying ranges of values and some specific to dealing with dates and times. The whole list is available in the Django documentation: http://mng.bz/5lm7.

4.5 Modifying your data

You've created and queried data, but that's just not enough. Data changes over time, so you need to be able to edit and delete data. To demonstrate, create another `Musician` object:

```
>>> Musician.objects.create(first_name="Joseph", last_name="Trudeau",
➥ birth=date(1938, 2, 22))
<Musician: Musician(id=4, last_name=Trudeau)>
```

This new object has two problems: first, my dad was born on the 23rd, not the 22nd. Second, he doesn't qualify as a musician. I love him dearly, but his relationship to pitch is a challenging one. Deal with the first problem first: as I didn't store the result of the `.create()` in a variable, I need to run a query to get the object back. The `.get()` method of the query manager retrieves a single object but only if there is a single match. Run a query using `.get()`, giving `id=4` and store that in a variable this time:

```
>>> dad = Musician.objects.get(id=4)
>>> dad
<Musician: Musician(id=4, last_name=Trudeau)>
```

With the object in hand, you can alter the attributes on the object. This, on its own, is not sufficient, you are only changing the object in memory. The `.save()` method on your object executes the underlying SQL to sync your object up with the database. Change the `birth` value to the 23rd, and then call `.save()` to update the database:

```
>>> dad.birth
datetime.date(1938, 2, 22)
>>> dad.birth = date(1938, 2, 23)
>>> dad.save()
```

Querying the same object into a new variable proves the value in the database has changed:

```
>>> joseph = Musician.objects.get(first_name="Joseph")
>>> joseph.birth
datetime.date(1938, 2, 23)
```